



Zero-Budget Build · Ship AURA for ~USD 0 and scale on revenue



Yes — and the architecture I chose is the reason why. You can build, ship and sell AURA for **USD 0 up to your first paying customer**, then about **USD 120–250 per year**. No servers, no GPU rental, no paid APIs are needed to reach a sellable Windows product.

Why it is nearly free (this is strategy, not luck)

1. **Local-first desktop app** — all processing runs on the customer's machine, so your **marginal cost per user and per image is zero, forever**. Aftershoot and Imagen pay GPU money every time a customer imports a wedding. You pay nothing.
2. **Bring-your-own API key** — the customer's key pays for cloud reasoning. No backend, no bill, no liability.
3. **Fitted models, not giant neural nets** — exposure targets, WB priors, tone intent, culling weights and the whole personal style profile are statistical fits that train in **minutes on a laptop CPU**.

The honest caveat: free in *money* is not free in *time*. Budget **4–7 months** of focused solo work with a coding agent for a sellable V1.

The stack is 100 % free for commercial use

Rust, Tauri 2, React, wgpu, ONNX Runtime, SQLite, OpenCV (Apache-2.0), Icms2, PyTorch — all MIT/Apache/BSD/public-domain, zero cost, commercial use

allowed.



One catch: LibRaw is LGPL. Ship it as a **dynamically linked DLL/dylib** and you are compliant and free. Static-linking it into a closed-source app requires the paid commercial licence. Design for a DLL from day one — it costs nothing and removes the problem permanently. Also use **kamadak-exif** (BSD) for EXIF, not **exiv2** (GPL), and write XMP yourself — it is just XML.

The real trap: model licences, not software licences

A pretrained model can be free to download and **illegal to sell**. This is where most "free AI product" plans quietly break.

Need	Free AND commercially usable	Avoid (research-only)
Face detection	YuNet (OpenCV Zoo)	InsightFace pretrained weights
Face embedding	SFace (OpenCV Zoo, Apache-2.0)	ArcFace / InsightFace releases
Perceptual embedding	CLIP / OpenCLIP (MIT)	—
Segmentation	MobileSAM / SAM (Apache-2.0)	CelebAMask-HQ face-parsing weights
Denoise, deblur	train your own on synthetic noise	most published restoration weights
Generative fill	sibling-frame borrowing (your code), FLUX.1 schnell (Apache-2.0)	LaMa weights (Places2)
Datasets	your own archive , partner weddings, Open Images (CC BY)	WIDER FACE, FFHQ, CelebA

Two habits make this a non-issue: **heuristics first** (Laplacian variance for focus, gray-world for WB, dHash for duplicates — free, instant, explainable, good enough to ship), and **fitted models over trained models**.

Only three things actually cost money

Item	Real cost	Zero-cost workaround	When to actually pay
Windows code signing	~USD 10/month (Azure Trusted Signing) or 120–500/yr	Ship unsigned in alpha/closed beta; warn testers about the SmartScreen prompt	The day a stranger pays you
Apple Developer Program	USD 99/year	Launch Windows-only — that is where the NVIDIA GPUs and your CUDA speed advantage are	~USD 300 MRR
Training data + GPU	the moat	Your own archive + Kaggle's free GPU hours + the learning loop	Never — it gets cheaper as you grow

Free-tier infrastructure map

Need	Free tool	Free limit	Paid successor
CI	GitHub Actions	2,000 min/mo private	USD 4/mo (macOS burns minutes 10×)
Model + installer hosting	Cloudflare R2	10 GB, zero egress fees	USD 0.015/GB/mo
Website + docs	Cloudflare Pages	unlimited bandwidth	—
Licence key server	Cloudflare Workers + D1	100k requests/day	USD 5/mo
Payments	Lemon Squeezy / Paddle	no fixed fee, ~5 % + 0.50	scales with revenue
Crash reports	Sentry	5k errors/mo	USD 26/mo
Analytics	PostHog	1M events/mo	usage-based
Email	Resend	3,000/mo	USD 20/mo
Training GPU	Kaggle Notebooks	~30 GPU-hours/week	rent hourly later
Domain	Cloudflare Registrar	~USD 10/year	—

Fixed cost to run a live, paid product on this map: about USD 10 per year until you buy code signing.

Your unfair advantage costs nothing

You run a wedding studio. You already own the most expensive asset in this business: **thousands of real weddings with RAW originals and photographer-approved finals**. A funded competitor has to buy that; you have it on a hard drive.

- Phase 17 is written to fit edit recipes from **JPEG finals alone**, so archives without XMP still work.
- **20 of your own weddings**, properly labelled, get every model in the plan to its quality gate.
- Nepali, Hindu and mixed-tradition weddings are exactly where Aftershoot and Imagen are weakest — your data is not just free, it is *differentiated*.

Do this month, costs nothing: archive 20 finished weddings as RAW + final + your delivery decisions.

Build 16 phases, not 30, before you charge

Lean V1	Phases
Foundation, RAW decode, inference runtime	01, 02, 03
Embeddings, bursts, duplicates	05, 08
Frame integrity (focus, motion, eyes)	09
People + scene at heuristic depth	06, 07 (light)
Autonomous culling + Explain My Edit	12, 13
Develop engine, exposure, WB, tone, colour	14, 15, 16
Personal style profiles	17
Autopilot button, export, XMP, learning loop	28, 30

That is sellable: *"Import 3,000 RAWs, click once, get a culled and graded gallery in your own style, with XMP for Lightroom."* At **USD 19–29/month** it competes head-on with the culling tools — with no retouching, no generative AI and no cloud.

Phases 18–27 and 29 become V2/V3, funded by V1 revenue.

The spend ladder — buy only when revenue triggers it

Trigger	Spend
Day 0	USD 0 — build, alpha with 3 photographer friends, unsigned installer
First beta users	~USD 10/year — domain
First paying customer	~USD 10/month — code signing (removes the SmartScreen warning that kills conversion)
~USD 300 MRR	USD 99/year — Apple Developer, unlock macOS
~USD 500 MRR	USD 4–26/month — CI minutes, Sentry
~USD 1,000 MRR	~USD 600–800 once — used RTX 3090/4070 for training
~USD 3,000 MRR	usage-based — rented cloud GPU for optional heavy features, resold as credits
Never	servers for core processing — local-first is the moat, do not give it away

Why your unit economics beat funded competitors

	Cloud competitor	AURA
Cost per 3,000-image wedding	GPU + storage + egress	0
Customer processing 40 weddings/year	scales linearly	0
Cloud reasoning cost	on their P&L	on the customer's own key
Gross margin	shrinks with heavy users	~89 %, flat
Break-even customers at USD 29/mo	dozens, with burn	1

A heavy user is a **cost** to them and **pure profit** to you. That is why you can price lower, offer unlimited weddings, and never need funding to survive growth.

Five rules that keep the cost at zero

1. **No servers in the critical path.** If it cannot finish a wedding with the network unplugged, that is a design bug.

2. **The customer's key pays for the customer's AI.** Never proxy their cloud calls through your infrastructure.
3. **Heuristic first, fitted second, neural last.**
4. **Licence-check every dependency and every model before merge.** Add it to the PR checklist.
5. **Every purchase needs a revenue trigger.** If you cannot name the customer whose payment justifies it, do not buy it.

What not to cheap out on

- **Code signing, once you charge money** — a scary Windows warning on a paid download is the most expensive USD 10 you will ever save.
- **Backups of your training archive** — your only irreplaceable asset. Two local, one offsite.
- **Colour correctness** — getting skin wrong is the one bug wedding photographers never forgive.
- **Honesty in marketing** — no funding is fine; overclaiming what the AI does is what kills products in this market.

The full version of this document ships in the bundle as `11-ZERO-BUDGET-BUILD.md`.

[AURA-Wedding-AI-Master-Plan.zip](#)

[AURA-Wedding-AI-Master-Plan.zip](#) — updated bundle with the zero-budget plan